

spec.md

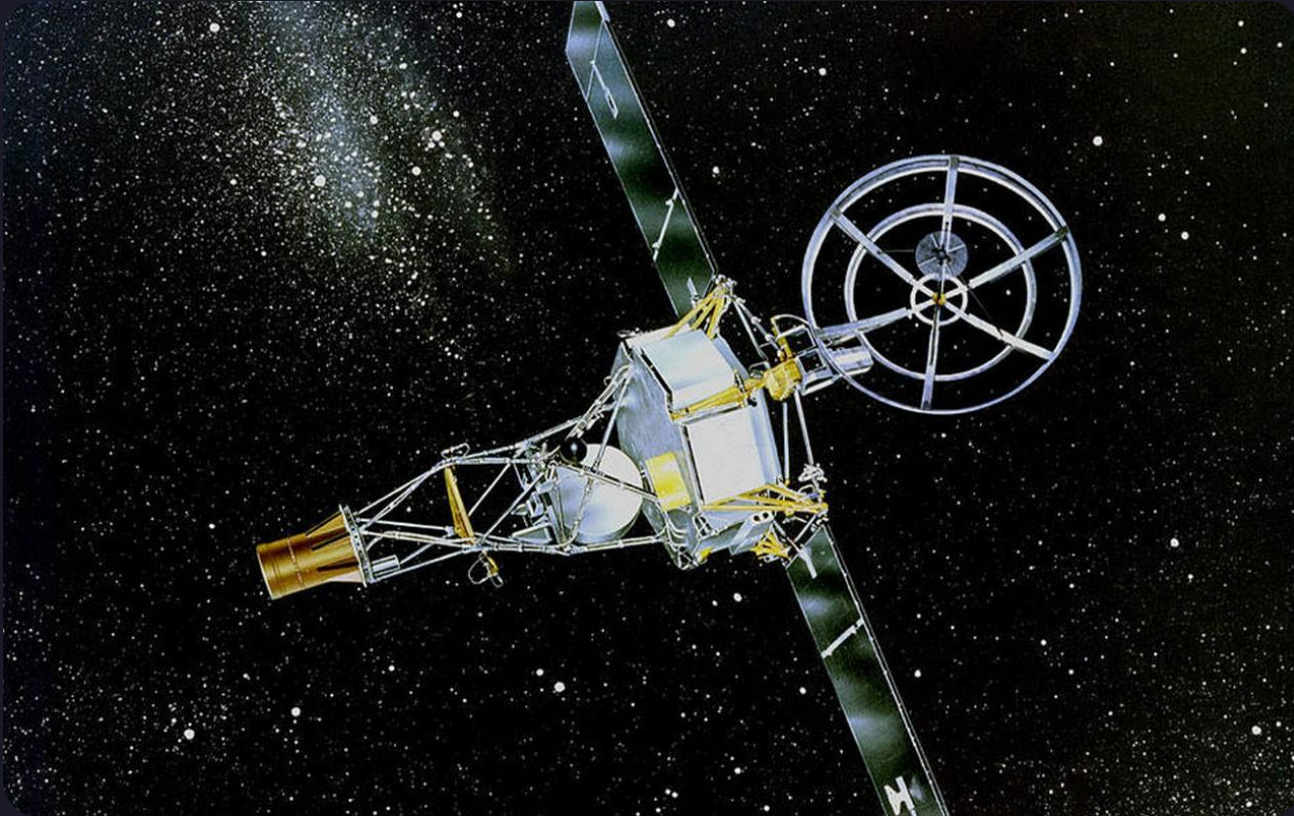
Desarrollo dirigido por especificaciones

SDD, GitHub Spec Kit y una demostración con cli-sdd

\$ intención → especificación → código

Mariner 1 y el comportamiento ante fallos

22 de julio de 1962 · La misión terminó 294,5 segundos después del lanzamiento



QUÉ OCURRIÓ

Un fallo de guía hizo que el vehículo se desviara de su trayectoria.

El incidente se relaciona con un error en una fórmula y problemas en los datos de radar. La respuesta ante esa situación anómala resultó insuficiente.

LA LECCIÓN

Los casos de error también forman parte del comportamiento que debe definirse antes de implementar.

El incidente muestra el coste de dejar una situación anómala sin una respuesta suficientemente definida.

Margaret Hamilton: ingeniería de software

Su trabajo llevó rigor, responsabilidad y previsión al desarrollo de software crítico



Fotografía completa · 1969

01 · DÉCADA DE 1960

Ayudó a formalizar la ingeniería de software

Hamilton defendió que programar debía tratarse con la misma seriedad que construir un puente o un avión. El software necesitaba diseño deliberado, pruebas rigurosas y responsabilidad sobre los fallos.

02 · 20 JUL 1969 · APOLLO 11



Alarmas 1201 y 1202

Ante la sobrecarga, el software prioriza las tareas esenciales y descartó trabajo menos importante. El descenso pudo continuar.

El nuevo cuello de botella del desarrollo con IA

La IA acelera el código.

La ambigüedad sigue siendo cara.

Un agente puede producir una solución técnicamente válida para un problema mal definido.

VIBE CODING

Útil para explorar y prototipar. Exige más control cuando la mantenibilidad, la seguridad o la escala importan.

Prompt aislado

Contexto parcial



Supuestos invisibles

La IA completa los huecos



Retrabajo

Código correcto, intención equivocada

Qué es Spec-Driven Development

Una especificación estructurada se convierte en el principal artefacto de comunicación

LA ESPECIFICACIÓN

Define qué debe hacer el sistema y por qué importa.



EL CÓDIGO

Materializa cómo se construye la solución.

SDD no busca producir documentación por formalidad. La especificación debe orientar decisiones y permitir validar el resultado.

Partes de una especificación

Ejemplo de las notas: subida de documentos PDF y DOCX

spec.md

Cada sección responde una pregunta distinta y añade condiciones que pueden verificarse antes de implementar.

El detalle técnico pertenece al plan salvo que represente una restricción ya acordada.

01	Resumen ¿Qué valor aporta?	Los empleados suben PDF o DOCX y los documentos aparecen en su panel.
02	Historia de usuario ¿Quién lo necesita?	Como empleado, quiero subir documentos para acceder a ellos desde cualquier dispositivo.
03	Criterios de aceptación ¿Cómo sabemos que funciona?	Máximo 50 MB, solo rol Contributor y aparición en la lista en un máximo de 2 segundos.
04	Requisitos funcionales ¿Qué debe hacer?	Validar tipo y tamaño, recibir el archivo y guardar sus metadatos.
05	Requisitos no funcionales ¿Con qué calidad?	TLS 1.2, objetos privados y subida menor de 5 MB en menos de 5 segundos.
06	Casos límite ¿Qué puede salir mal?	Red interrumpida, MIME incorrecto, nombre duplicado o exceso de cargas simultáneas.

Del ciclo clásico al flujo asistido por IA

SDD adapta etapas conocidas y convierte sus resultados en contexto persistente

CICLO CLÁSICO



La diferencia práctica: cada etapa deja un artefacto que la siguiente puede leer, revisar y versionar.

Tres formas de relacionar especificación y código

La diferencia aparece cuando el software cambia después de la primera implementación

Menor compromiso

Mayor compromiso

Spec-first

La especificación inicia el trabajo. Después, el código puede convertirse en la única fuente de verdad.

Fuente final: código

Spec-anchored

Código y especificación evolucionan juntos. Un cambio en cualquiera exige revisar el otro.

Fuente final: ambos

Spec-as-source

La especificación conserva la autoridad y permite generar o regenerar la implementación.

Fuente final: especificación

La pregunta clave: ¿qué ocurre con la especificación cuando el código ya existe?

Desde la intención hasta una revisión verificable

Cada artefacto reduce un tipo diferente de ambigüedad



La conversación deja de depender del historial del chat. La intención queda registrada en archivos que Git puede revisar.

GitHub Spec Kit convierte el flujo en artefactos y comandos

Los archivos viven en el repositorio y alimentan cada fase del agente

constitution.md

Principios y restricciones

spec.md

Requisitos y criterios de aceptación

plan.md

Arquitectura y decisiones técnicas

tasks.md

Trabajo concreto y secuenciado

• • • agent workflow

01 /speckit.constitution

06 /speckit.tasks

02 /speckit.specify

07 /speckit.analyze

03 /speckit.clarify

08 /speckit.implement

04 /speckit.plan

09 /speckit.converge

05 /speckit.checklist

quality gates

clarify · checklist · analyze

La versión actual también incorpora extensiones, presets y workflows para adaptar o automatizar el proceso.

El proyecto completo está en GitHub

La explicación continúa directamente desde los artefactos, el código y las pruebas



cli-sdd

CLI de tareas con Node.js, TypeScript, Prisma y SQLite

github.com/sergio-pomarico/cli-sdd

Haz clic en el enlace para abrir el repositorio

[constitution.md](#) [spec.md](#) [plan.md](#) [tasks.md](#) [código](#) [pruebas](#)